Evaluación Parcial N.º 2

Implementación de bases de datos NoSQL y manipulación de datos

Caso QuickTrack
Azure Cosmos DB for MongoDB (RU)

<table>
<tr><td colspan="2">Sección: 002D</td><td rowspan="2">Bases de Datos
No Relacionales</td></tr>
<tr><td>Fecha de elaboración:
08/11/2026</td><td>Docente:
Igor Venegas Morales</td></tr>
<tr><td colspan="3">Integrantes:
1.- Anyara Rosso
2.- Italo Zapata</td></tr>
</table>

Índice

1. Objetivos del proyecto 3
    1.1 Descripción del proyecto 3
    1.2 Objetivo general 3
    1.3 Objetivos específicos 4
2. Configuración de la base de datos NoSQL 5
    2.1 APIs de Azure Cosmos DB 5
    2.2 Elección de la API 5
    2.3 Creación del recurso 5
    2.4 Decisiones de configuración 7
3. Inserción de los datos 8
    3.1 Creación de la base de datos y las colecciones 8
    3.2 Preparación de mongoimport 9
    3.3 Carga de los datos 10
    3.4 Verificación 11
4. Operaciones de lectura 13
    4.1 Revisión previa de los datos 13
    4.2 Consulta 1: Órdenes pendientes de alto valor 13
    4.3 Eficiencia de la consulta 1 14
    4.4 Consulta 2: Órdenes en efectivo enviadas en 2023 y entregadas 15
    4.5 Consulta 3: Ventas por medio de pago 15
    4.6 Consulta 4: Órdenes canceladas por zona 16
    4.7 Consulta 5: Satisfacción según el comentario 17
5. Escritura, actualización y eliminación 19
    5.1 Revisión 1: Órdenes sin cliente 19
    5.2 Revisión 2: País mal escrito 20
    5.3 Revisión 3: Estado actual distinto al historial 20
    5.4 Caso 1: Registrar un cliente nuevo y su primera orden 21
    5.5 Caso 2: La orden sale a reparto 22
    5.6 Caso 3: Registrar la misma orden otra vez 22
    5.7 Caso 4: El cliente anula la orden 23
6. Integración con Python 24
    6.1 Conexión desde Colab 24
    6.2 Funciones CRUD 24
7. Trazabilidad y control de errores 25
    7.1 Registro de operaciones 25
    7.2 Validaciones y manejo de errores 25
    7.3 Archivo de log 26

1. Objetivos del proyecto

1.1 Descripción del proyecto

El presente proyecto tiene como finalidad implementar y utilizar una base de datos no relacional mediante Azure Cosmos DB for MongoDB, tomando como referencia el caso de negocio de QuickTrack, una empresa de logística que ofrece servicios de entrega express a nivel nacional.

La empresa cuenta con un sistema digital que almacena información relacionada con las órdenes de entrega, los clientes, las rutas utilizadas, los seguimientos de los paquetes y las valoraciones realizadas por los usuarios.

Debido al volumen de información generado por sus operaciones, QuickTrack requiere una solución que permita almacenar, consultar y manipular los datos de manera eficiente, facilitando el análisis de su funcionamiento logístico.

Para abordar esta necesidad, se utilizará MongoDB como modelo de base de datos documental, permitiendo organizar la información mediante colecciones y documentos JSON/BSON. Esta implementación permitirá realizar operaciones de lectura, inserción, actualización y eliminación de datos (CRUD), además de facilitar futuras integraciones con aplicaciones desarrolladas en Python.

Finalmente, los datos almacenados podrán utilizarse para desarrollar análisis orientados a identificar zonas críticas de retraso, evaluar la satisfacción de los clientes y apoyar la optimización de las rutas de distribución, contribuyendo a la toma de decisiones estratégicas de la empresa.

1.2 Objetivo general

Implementar una base de datos no relacional utilizando Azure Cosmos DB for MongoDB, que permita almacenar, administrar y consultar la información operacional de la empresa QuickTrack, mediante operaciones CRUD y consultas sobre documentos, facilitando el análisis de los procesos logísticos y proporcionando información útil para la toma de decisiones empresariales.

1.3 Objetivos específicos

- Configurar una cuenta de Azure Cosmos DB utilizando la API compatible con MongoDB, estableciendo los parámetros necesarios para su funcionamiento.
- Diseñar y organizar la estructura de la base de datos mediante colecciones que permiten almacenar información de clientes, órdenes de entrega, rutas, seguimientos y valoraciones.
- Incorporar los conjuntos de datos proporcionados para el caso QuickTrack, verificando que la información se almacene correctamente y que exista conectividad con la base de datos.
- Desarrollar consultas de lectura utilizando filtros, proyecciones, ordenamientos y operadores de MongoDB para recuperar información relevante según las necesidades del negocio.
- Implementar operaciones de inserción, actualización y eliminación de documentos, validando los resultados y evitando inconsistencias o duplicación de información.
- Integrar la base de datos con una aplicación desarrollada en Python mediante bibliotecas compatibles con MongoDB, permitiendo realizar operaciones de lectura y escritura.
- Generar información que permita analizar el desempeño de las operaciones logísticas, considerando las rutas utilizadas, los estados de los pedidos y las valoraciones de los clientes.

2. Configuración de la base de datos NoSQL

2.1 APIs de Azure Cosmos DB

Azure Cosmos DB ofrece varias APIs. Cada una usa un modelo de datos distinto:

- API para NoSQL (Core SQL): es la API nativa de Cosmos DB. Guarda documentos JSON y los consulta con una sintaxis parecida a SQL.
- API para MongoDB: es compatible con el protocolo de MongoDB. Permite usar herramientas como mongosh, MongoDB Compass, mongoimport y librerías como pymongo.
- API para Apache Cassandra: usa un modelo de columnas anchas y se consulta con CQL. Sirve para grandes volúmenes de escritura.
- API para Apache Gremlin: trabaja con grafos, es decir, nodos y relaciones. Sirve para redes sociales o sistemas de recomendación.
- API para Table: guarda datos clave-valor. Es compatible con Azure Table Storage.

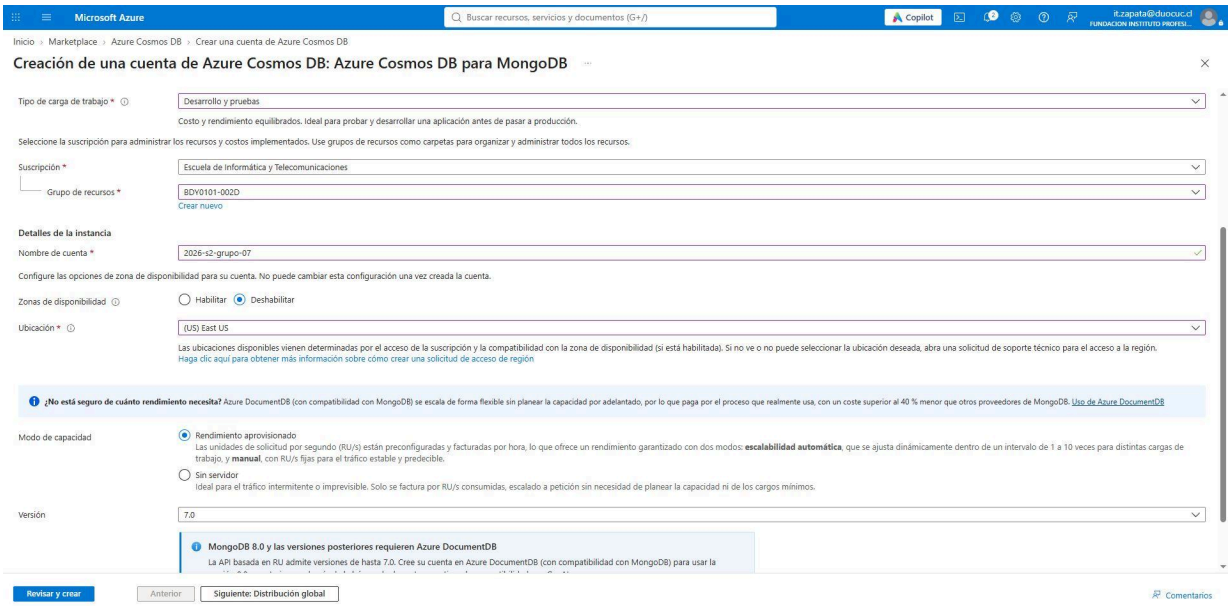
2.2 Elección de la API

Elegimos la API para MongoDB por estas razones:

- Los datos de QuickTrack vienen en archivos JSON, que calzan con un modelo de documentos.
- El seguimiento tiene un historial guardado como arreglo dentro de cada documento. El modelo documental lo maneja sin crear tablas aparte.
- Podemos usar las herramientas del caso: mongoimport para la carga, Compass para las consultas y pymongo para la integración con Python.

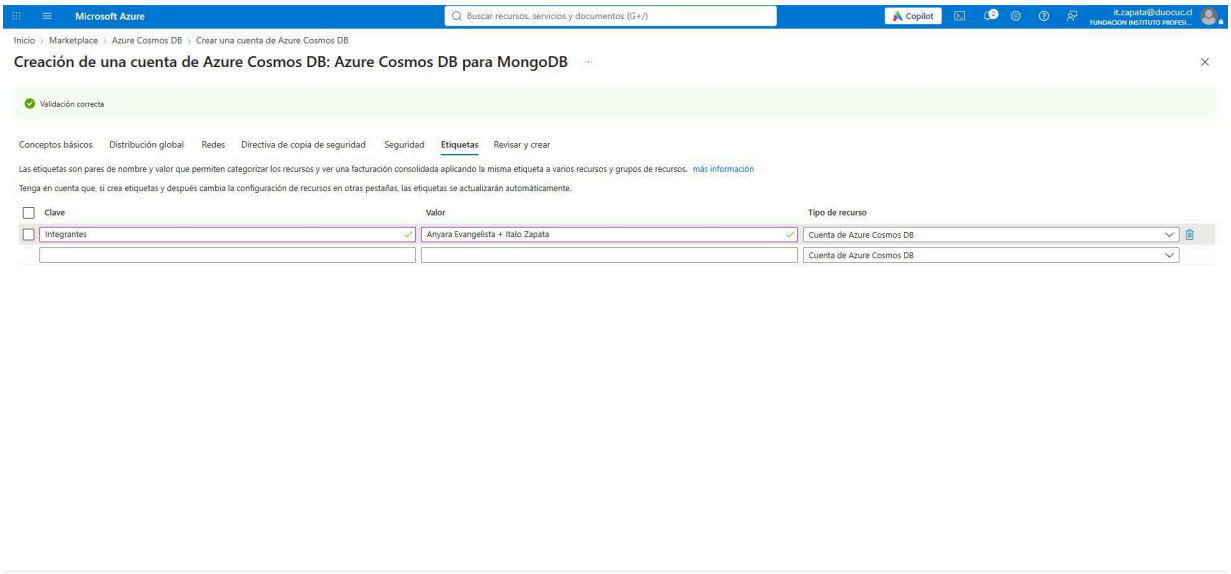
2.3 Creación del recurso

En esta primera parte definimos las configuraciones del recurso.



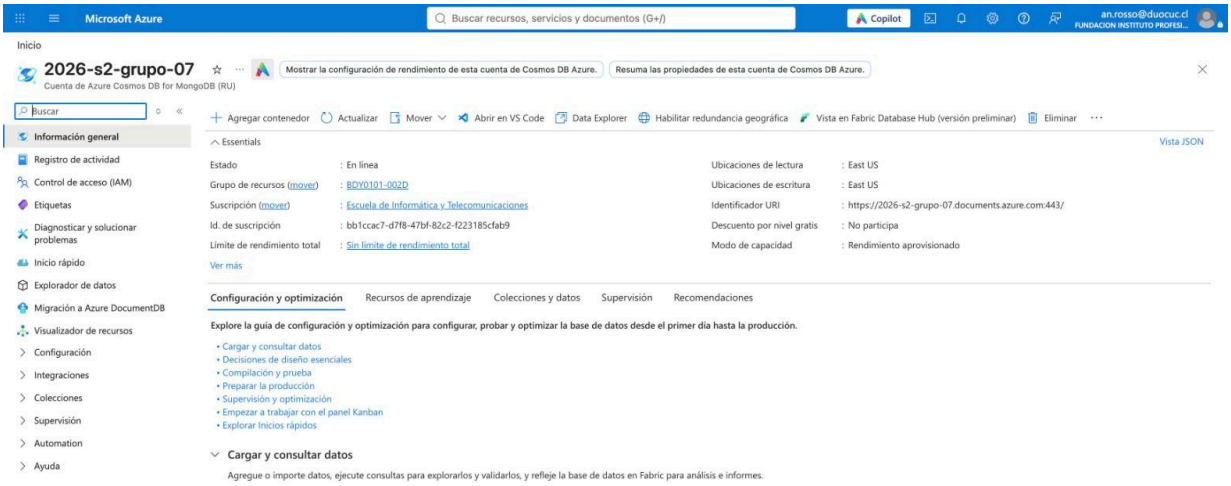
- Tipo de carga de trabajo: Desarrollo y pruebas
- Suscripción: Escuela de Informática y Telecomunicaciones
- Grupo de recursos: BDY0101-002D
- Nombre de la cuenta: 2026-s2-grupo-07
- Ubicación: (US) East US
- Modo de capacidad: Rendimiento aprovisionado
- Versión de MongoDB: 7.0

Definimos las etiquetas del recurso.



- Clave: Integrantes
- Valor: Anyara Evangelista + Italo Zapata

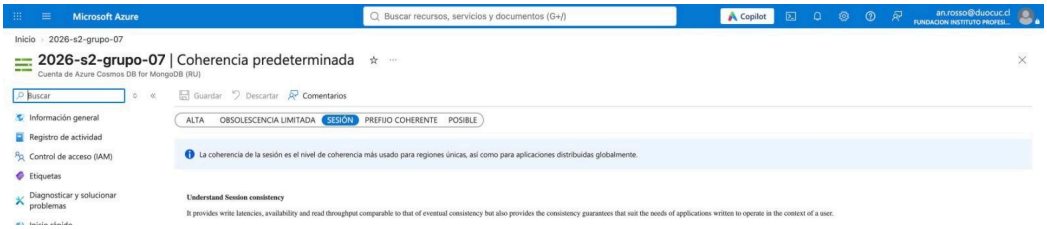
En esta imagen se puede ver el recurso creado y sus opciones. En la información general aparece el modo de capacidad Rendimiento aprovisionado, la ubicación East US y el grupo de recursos BDY0101-002D.



2.4 Decisiones de configuración

- Contenedores: en la API para MongoDB cada contenedor es una colección. Creamos cinco colecciones, una por cada archivo del caso: clientes, ordenes-entrega, rutas, seguimientos y valoraciones.
- Claves de partición: las colecciones son Unsharded, sin clave de partición y con un límite de 20 GB, suficiente para 100 documentos por colección. Si los datos crecieran, usaríamos cliente_id en ordenes-entrega y orden_id en seguimientos y valoraciones.
- Throughput: 400 RU/s manuales a nivel de base de datos, compartidos por las cinco colecciones. Es el mínimo que permite Azure y alcanza para el volumen del caso.
- Consistencia: se mantuvo la predeterminada, Sesión. Cada conexión lee sus propias escrituras, lo que basta para una cuenta de una sola región.
- Credenciales: la conexión usa la cadena de conexión principal de la cuenta, por el puerto 10255 y con SSL.

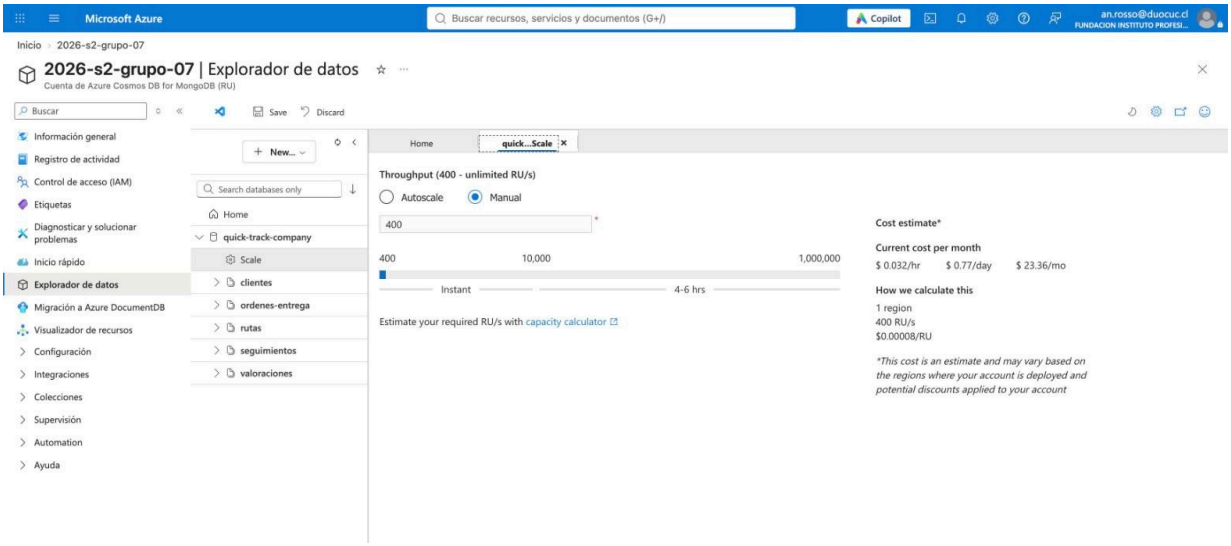
Configuración de la consistencia de la cuenta:



3. Inserción de los datos

3.1 Creación de la base de datos y las colecciones

En el siguiente paso creamos la base de datos y sus colecciones en el explorador de datos.



Nombre de la base de datos: quick-track-company

Colecciones:

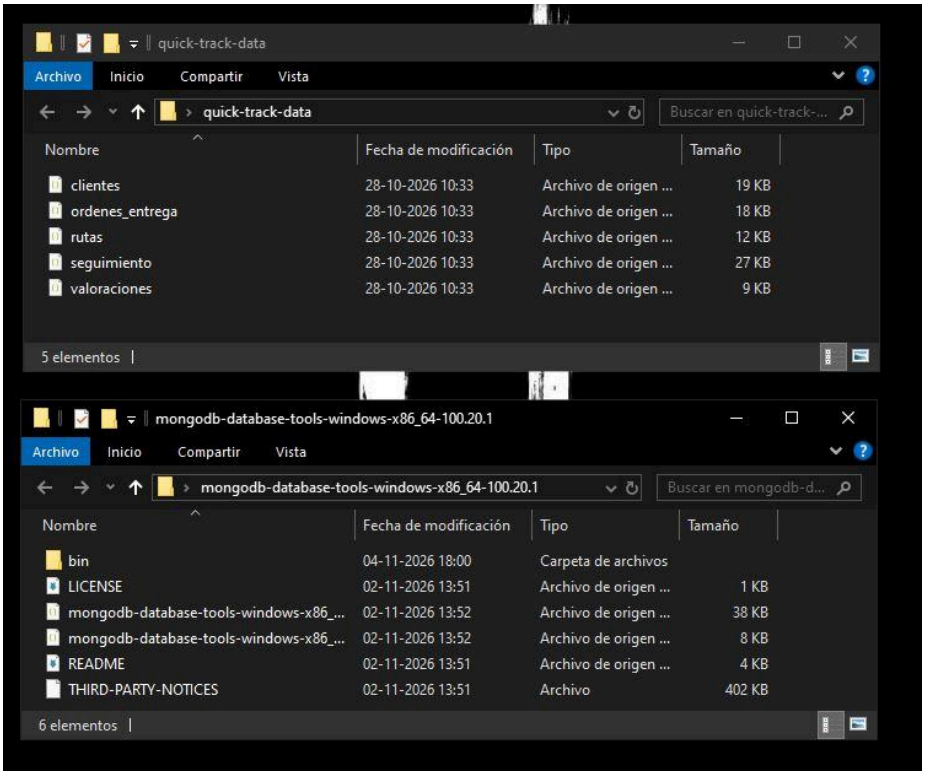
- clientes
- ordenes-entrega
- rutas
- seguimientos
- valoraciones

Configuración de las colecciones:

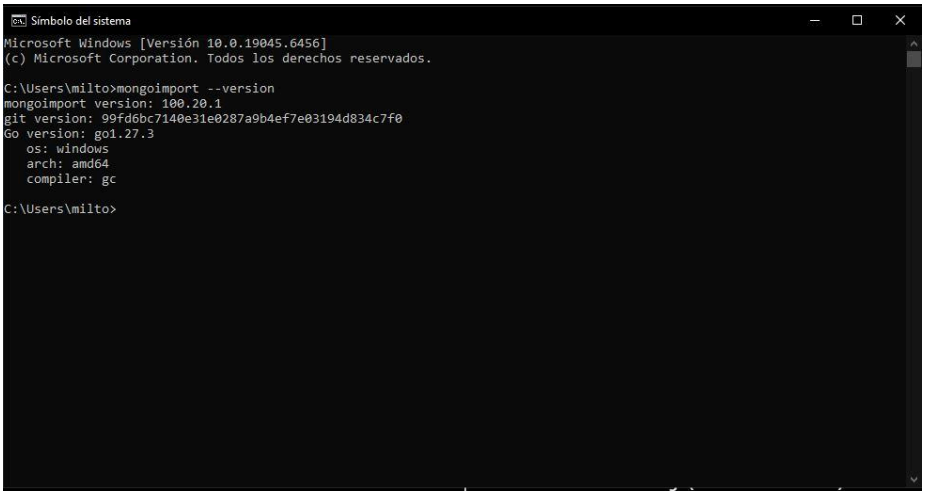
- Sharding: Unsharded.
- Límite: 20 GB.
- Rendimiento: 400 RU/s manuales, compartidos entre las cinco colecciones.

3.2 Preparación de mongoimport

Para la importación de datos tenemos que usar la herramienta mongoimport. La descargamos desde la página oficial, junto con los datos de las colecciones en formato JSON.

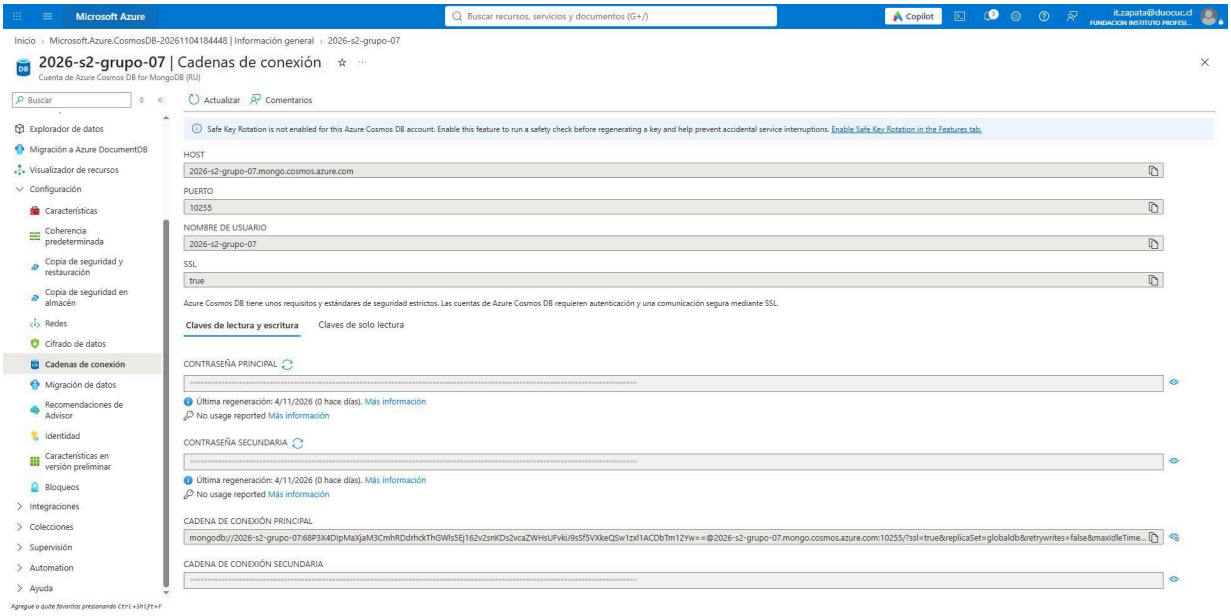


Verificamos que mongoimport funcione en la consola del computador consultando su versión.



En caso de que no salga ese mensaje en la consola, hay que revisar las variables de entorno del computador y añadir la ruta del archivo.

Ahora obtenemos la cadena de conexión principal del recurso para hacer la conexión con mongoimport.



3.3 Carga de los datos

Después, en la consola, importamos los datos con un comando por cada colección, indicando:

- uri: la cadena de conexión, guardada en la variable COSMOS_URI.
- db: la base de datos, quick-track-company.
- collection: el nombre de la colección.
- file: la ruta del archivo JSON de la colección.

Además usamos estas opciones:

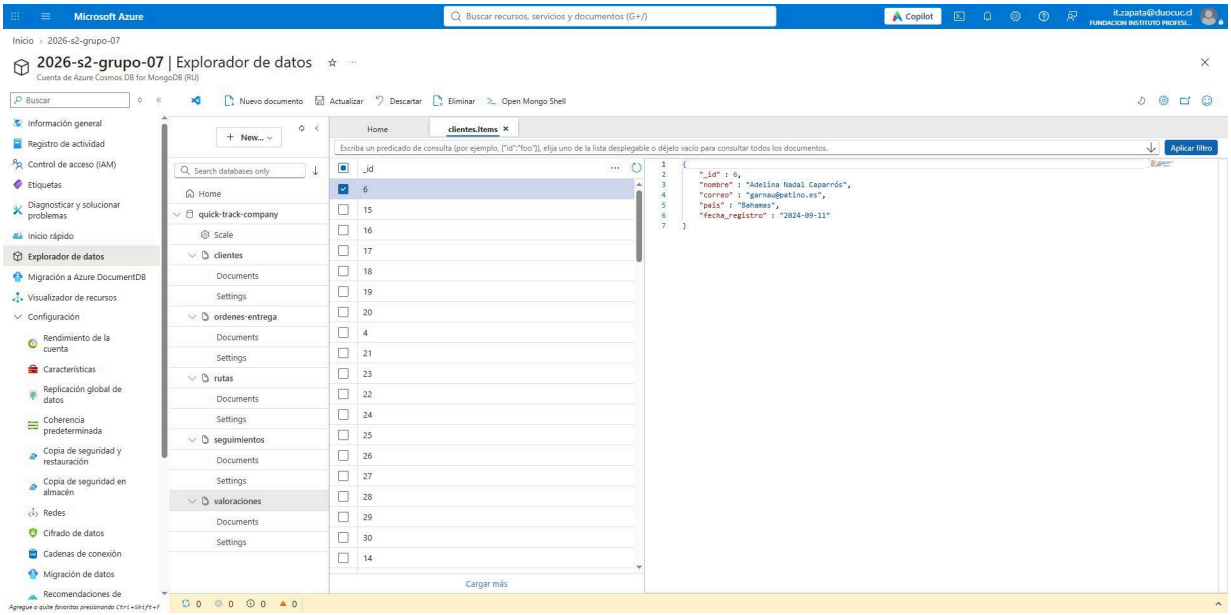
- jsonArray, solo en clientes, porque ese archivo es un arreglo JSON.
- numInsertionWorkers y batchSize en 1, para no superar los 400 RU/s durante la carga.

```
quick-track-data — -zsh — 150×62
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection clientes --file clientes.json --jsonArray --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:29:15.565-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:29:18.569-0300    [################........] quick-track-company.clientes 13.4KB/18.4KB (72.7%)
2026-11-04T22:29:21.573-0300    [####################..] quick-track-company.clientes 17.3KB/18.4KB (93.8%)
2026-11-04T22:29:24.575-0300    [########################] quick-track-company.clientes 18.4KB/18.4KB (100.0%)
2026-11-04T22:29:27.577-0300    [########################] quick-track-company.clientes 18.4KB/18.4KB (100.0%)
2026-11-04T22:29:28.834-0300    [########################] quick-track-company.clientes 18.4KB/18.4KB (100.0%)
2026-11-04T22:29:28.834-0300    100 document(s) imported successfully. 0 document(s) failed to import.
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection ordenes-entrega --file ordenes_entrega.json --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:29:45.667-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:29:48.668-0300    [################........] quick-track-company.ordenes-entrega  12.0KB/17.2KB (69.9%)
2026-11-04T22:29:51.669-0300    [####################..] quick-track-company.ordenes-entrega  16.0KB/17.2KB (93.2%)
2026-11-04T22:29:54.669-0300    [########################] quick-track-company.ordenes-entrega  17.2KB/17.2KB (100.0%)
2026-11-04T22:29:57.669-0300    [########################] quick-track-company.ordenes-entrega  17.2KB/17.2KB (100.0%)
2026-11-04T22:29:58.810-0300    [########################] quick-track-company.ordenes-entrega  17.2KB/17.2KB (100.0%)
2026-11-04T22:29:58.810-0300    100 document(s) imported successfully. 0 document(s) failed to import.
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection rutas --file rutas.json --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:30:15.899-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:30:18.900-0300    [################........] quick-track-company.rutas    8.00KB/11.9KB (67.4%)
2026-11-04T22:30:21.900-0300    [####################....] quick-track-company.rutas    10.9KB/11.9KB (91.5%)
2026-11-04T22:30:24.900-0300    [########################] quick-track-company.rutas    11.9KB/11.9KB (100.0%)
2026-11-04T22:30:27.900-0300    [########################] quick-track-company.rutas    11.9KB/11.9KB (100.0%)
2026-11-04T22:30:28.939-0300    [########################] quick-track-company.rutas    11.9KB/11.9KB (100.0%)
2026-11-04T22:30:28.939-0300    100 document(s) imported successfully. 0 document(s) failed to import.
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection seguimientos --file seguimiento.json --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:30:46.073-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:30:49.073-0300    [################........] quick-track-company.seguimientos     17.4KB/26.4KB (65.9%)
2026-11-04T22:30:52.073-0300    [####################....] quick-track-company.seguimientos     24.0KB/26.4KB (90.8%)
2026-11-04T22:30:55.074-0300    [########################] quick-track-company.seguimientos     26.4KB/26.4KB (100.0%)
2026-11-04T22:30:58.074-0300    [########################] quick-track-company.seguimientos     26.4KB/26.4KB (100.0%)
2026-11-04T22:30:59.452-0300    [########################] quick-track-company.seguimientos     26.4KB/26.4KB (100.0%)
2026-11-04T22:30:59.452-0300    100 document(s) imported successfully. 0 document(s) failed to import.
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection valoraciones --file valoraciones.json --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:31:16.206-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:31:16.633-0300    Failed: bulk write exception: write errors: [document does not contain shard key at 'id']
2026-11-04T22:31:16.633-0300    0 document(s) imported successfully. 1 document(s) failed to import.
any@any-mac quick-track-data % mongoimport --uri "$COSMOS_URI" --db quick-track-company --collection valoraciones --file valoraciones.json --numInsertionWorkers 1 --batchSize 1
2026-11-04T22:32:26.149-0300    connected to: mongodb://[**REDACTED**]@2026-s2-grupo-07.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retrywrites=false&maxIdleTimeMS=120000&appName=@2026-s2-grupo-07@
2026-11-04T22:32:29.151-0300    [################........] quick-track-company.valoraciones     6.98KB/8.82KB (79.1%)
2026-11-04T22:32:32.150-0300    [####################....] quick-track-company.valoraciones     8.00KB/8.82KB (90.7%)
2026-11-04T22:32:35.149-0300    [########################] quick-track-company.valoraciones     8.82KB/8.82KB (100.0%)
2026-11-04T22:32:38.150-0300    [########################] quick-track-company.valoraciones     8.82KB/8.82KB (100.0%)
2026-11-04T22:32:39.147-0300    [########################] quick-track-company.valoraciones     8.82KB/8.82KB (100.0%)
2026-11-04T22:32:39.147-0300    100 document(s) imported successfully. 0 document(s) failed to import.
any@any-mac quick-track-data %
```

Las cinco colecciones quedaron con 100 documentos cada una.

3.4 Verificación

Revisamos en el recurso que se hayan importado todos los datos.



Con esto comprobamos la conexión a la base desde tres herramientas: el explorador de datos de Azure, mongoimport y MongoDB Compass, que usamos en las consultas de lectura. En el apartado 6 se suma Python desde Google Colab.

4. Operaciones de lectura

Las consultas se ejecutaron en la consola mongosh de MongoDB Compass. Cada una responde a una necesidad del negocio.

4.1 Revisión previa de los datos

Antes de las consultas revisamos los datos y encontramos estas inconsistencias:

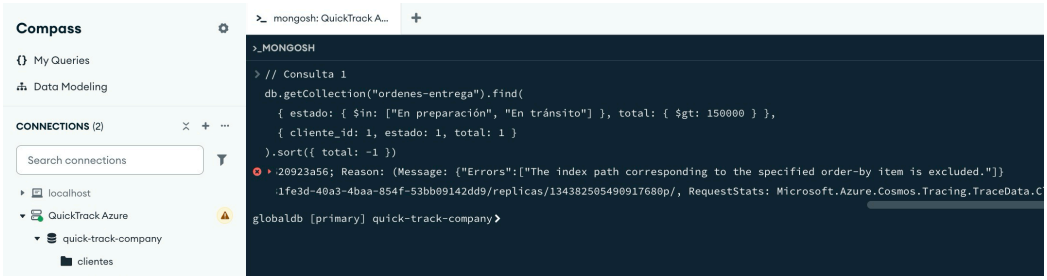
- El estado de la orden coincide con el estado_actual del seguimiento solo en 18 de 100 órdenes. Usamos el estado de la orden.
- No hay fecha de entrega, solo fecha_envio, guardada como texto.
- Hay valoraciones de órdenes que no se entregaron.

4.2 Consulta 1: Órdenes pendientes de alto valor

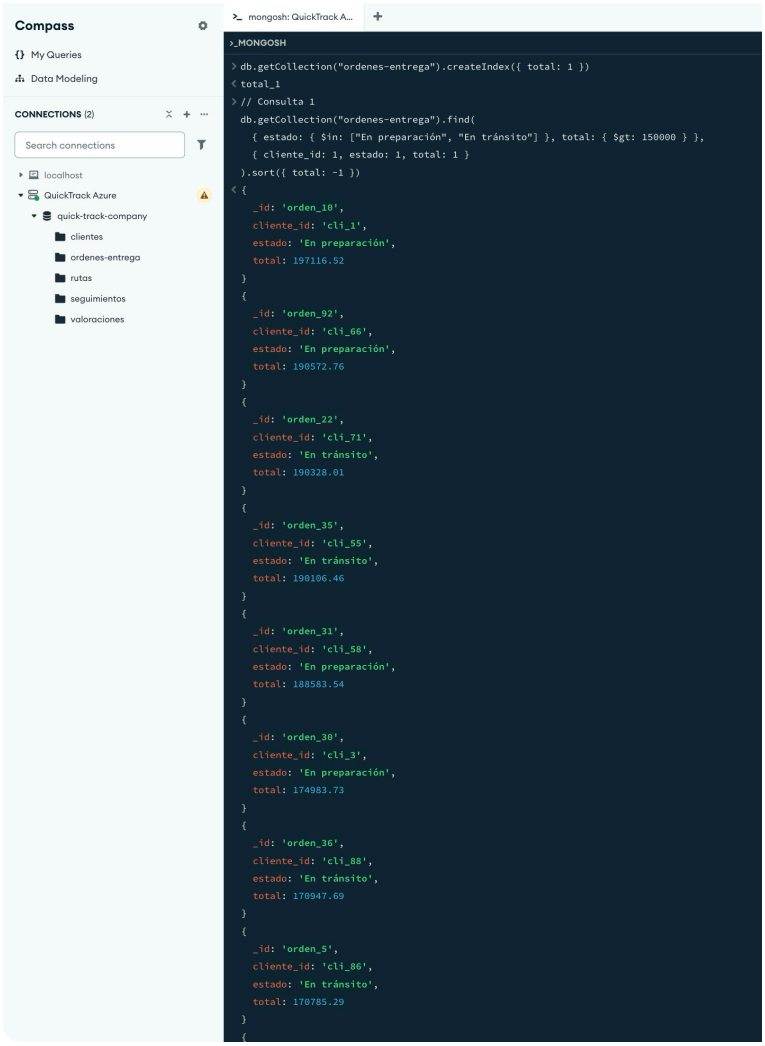
Necesidad: operaciones quiere revisar primero las órdenes que aún no se entregan y que tienen un total mayor a $150.000.

- Operadores: $in para los estados "En preparación" y "En tránsito", y $gt para el total.
- Proyección: solo cliente_id, estado y total.
- Ordenamiento: por total, de mayor a menor.

La primera vez dio error, porque Cosmos DB no permite ordenar por un campo sin índice.



Creamos un índice sobre total y la consulta devolvió 15 órdenes.



4.3 Eficiencia de la consulta 1

Medimos el costo de la consulta con getLastRequestStatistics:



- Costo: 9,04 RU.
- Tiempo de respuesta: 6 ms.
- Volumen devuelto: 15 documentos con 4 campos cada uno.

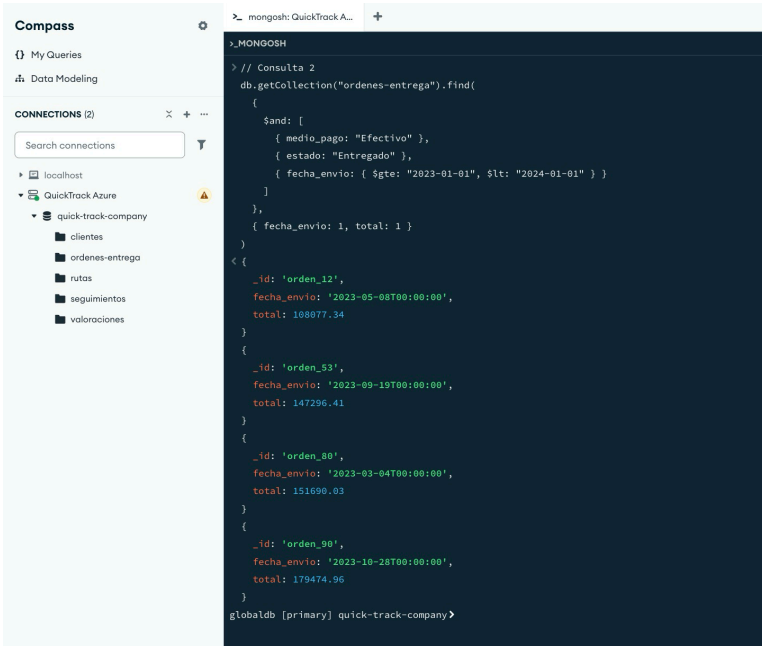
Mejoras propuestas:

- Mantener el índice sobre total.
- Si crece el volumen, agregar un índice sobre estado, el campo de filtro.
- Usar proyección para devolver solo los campos necesarios.

4.4 Consulta 2: Órdenes en efectivo enviadas en 2023 y entregadas

Necesidad: finanzas pide las órdenes pagadas en efectivo, enviadas en 2023 y que figuran como entregadas.

- Operadores: búsqueda compuesta con $and, más $gte y $lt para el rango de fechas.
- Proyección: fecha_envio y total.
- Observación: como fecha_envio es texto, el rango se compara como texto.

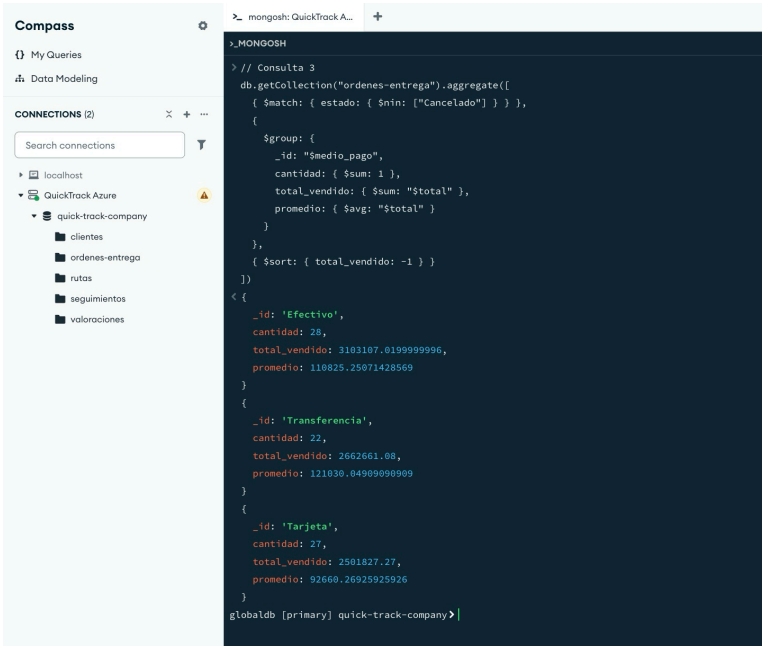


El resultado son 4 órdenes.

4.5 Consulta 3: Ventas por medio de pago

Necesidad: saber cuánto se vende con cada medio de pago y el monto promedio por orden.

- Operadores: $match, $group con $sum y $avg, y $sort.
- Criterio: se excluyen las órdenes canceladas.
- Observación: usamos $nin porque en Cosmos DB $ne no devolvió resultados dentro de aggregate.



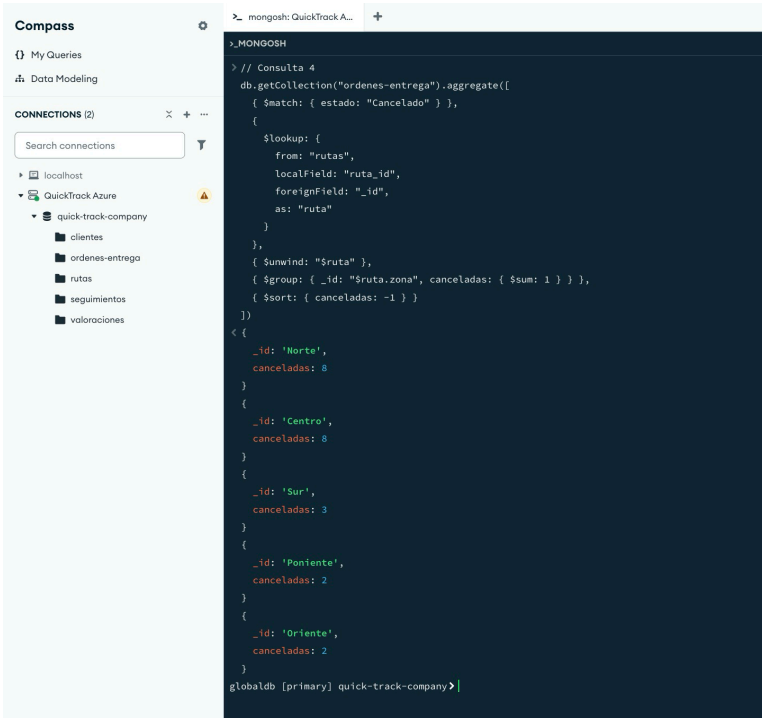
Medio de pago	Órdenes	Total vendido	Promedio
Efectivo	28	$3.103.107	$110.825
Transferencia	22	$2.662.661	$121.030
Tarjeta	27	$2.501.827	$92.660

Efectivo concentra el mayor total. Transferencia tiene el promedio más alto por orden.

4.6 Consulta 4: Órdenes canceladas por zona

Necesidad: encontrar las zonas con más problemas en las entregas.

- Operadores: $match, $lookup para unir las órdenes con las rutas, $unwind, $group y $sort.
- Observación: sin fecha de entrega no se puede medir el retraso, por eso usamos las cancelaciones como referencia.

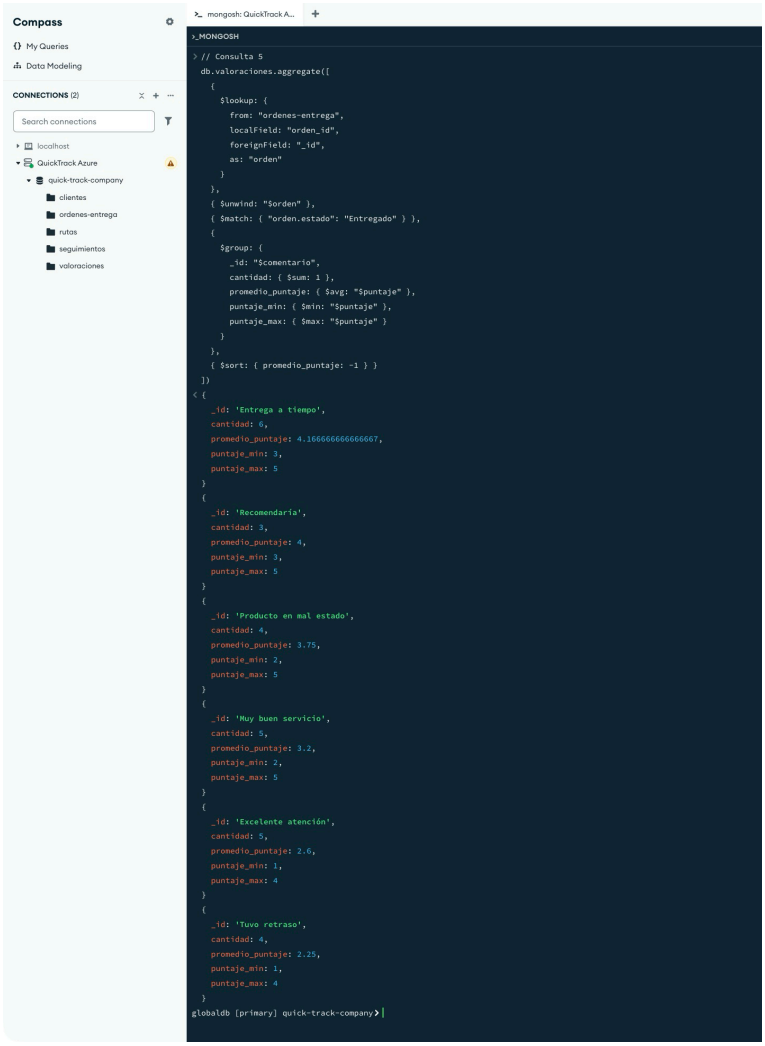


Norte y Centro concentran la mayor cantidad de cancelaciones, con 8 cada una.

4.7 Consulta 5: Satisfacción según el comentario

Necesidad: medir la satisfacción de los clientes (puntaje de 1 a 5) y revisar si el comentario coincide con el puntaje.

- Operadores: $lookup para unir las valoraciones con las órdenes, $match, $group con $avg, $min y $max, y $sort.
- Criterio: solo órdenes con estado "Entregado" (27 valoraciones).



El comentario no siempre coincide con el puntaje:

- "Excelente atención" tiene un promedio de 2,6.
- "Producto en mal estado" tiene un promedio de 3,75.

Por eso, para medir la satisfacción conviene usar el puntaje y no el comentario.

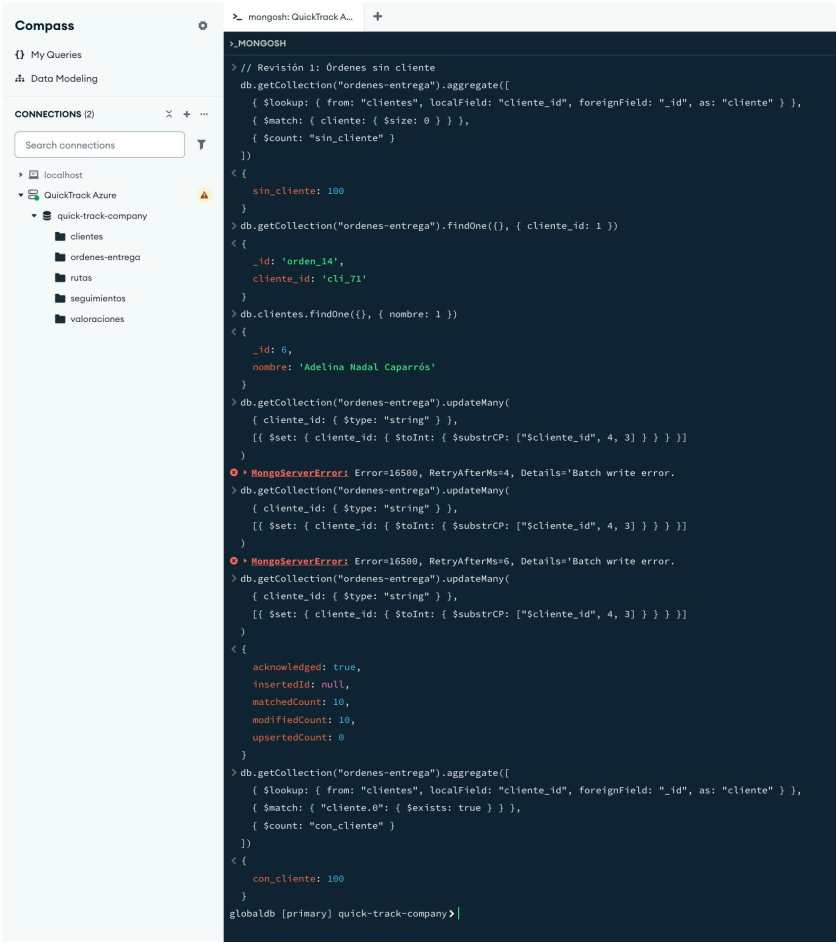
5. Escritura, actualización y eliminación

Las operaciones se ejecutaron en la consola mongosh de MongoDB Compass. Primero corregimos tres inconsistencias de los datos y después probamos cuatro casos del negocio.

No borramos ni inventamos datos. Lo que no se puede corregir, como las valoraciones de órdenes no entregadas, se manejó con filtros en el apartado 4.

5.1 Revisión 1: Órdenes sin cliente

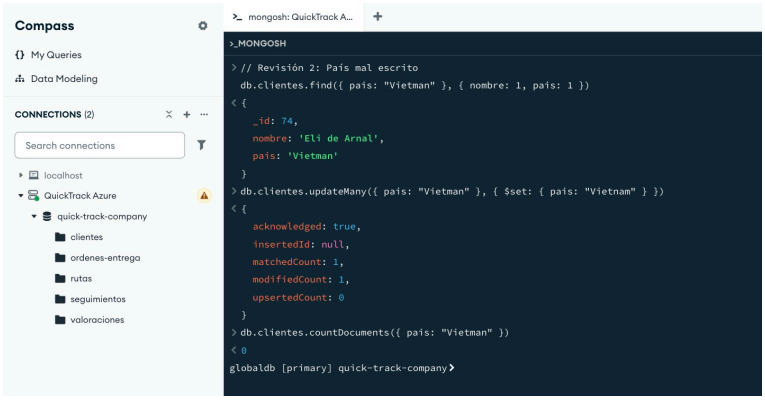
- Problema: con $lookup, ninguna de las 100 órdenes encuentra su cliente.
- Causa: en las órdenes, cliente_id es texto ("cli_71"), y en los clientes el _id es un número (6).
- Decisión: convertir cliente_id a número. Se cambian las órdenes porque el _id de los clientes no se puede modificar.
- Operadores: updateMany con $type, $substrCP y $toInt.



Cosmos DB cortó la actualización dos veces con el error 16500, por el límite de 400 RU/s. Al repetir el comando, siguió con las órdenes que faltaban. Al final, las 100 órdenes tienen cliente.

5.2 Revisión 2: País mal escrito

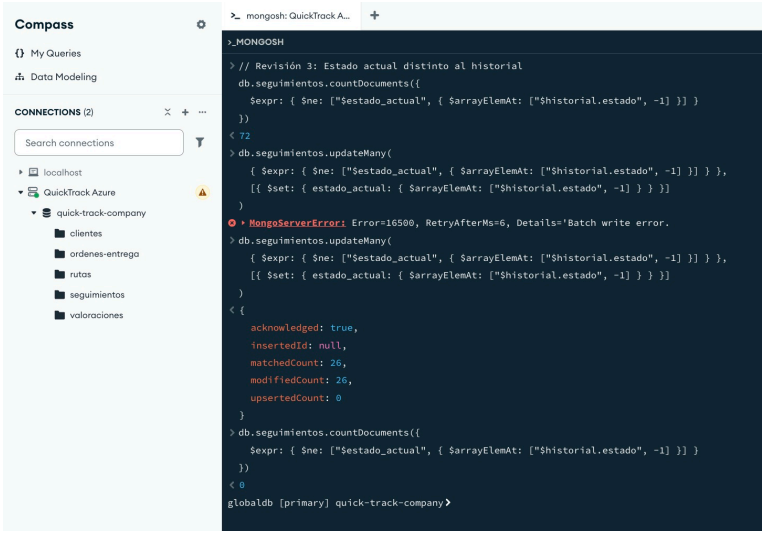
- Problema: el cliente 74 tiene el país escrito como "Vietman".
- Decisión: corregirlo a "Vietnam", para que no se cuente como otro país.



No quedan clientes con "Vietman".

5.3 Revisión 3: Estado actual distinto al historial

- Problema: en 72 de 100 seguimientos, estado_actual no coincide con el último estado del historial.
- Decisión: usar el último estado del historial, que está ordenado por fecha. No se borra nada del historial.
- Operadores: $expr, $ne y $arrayElemAt.



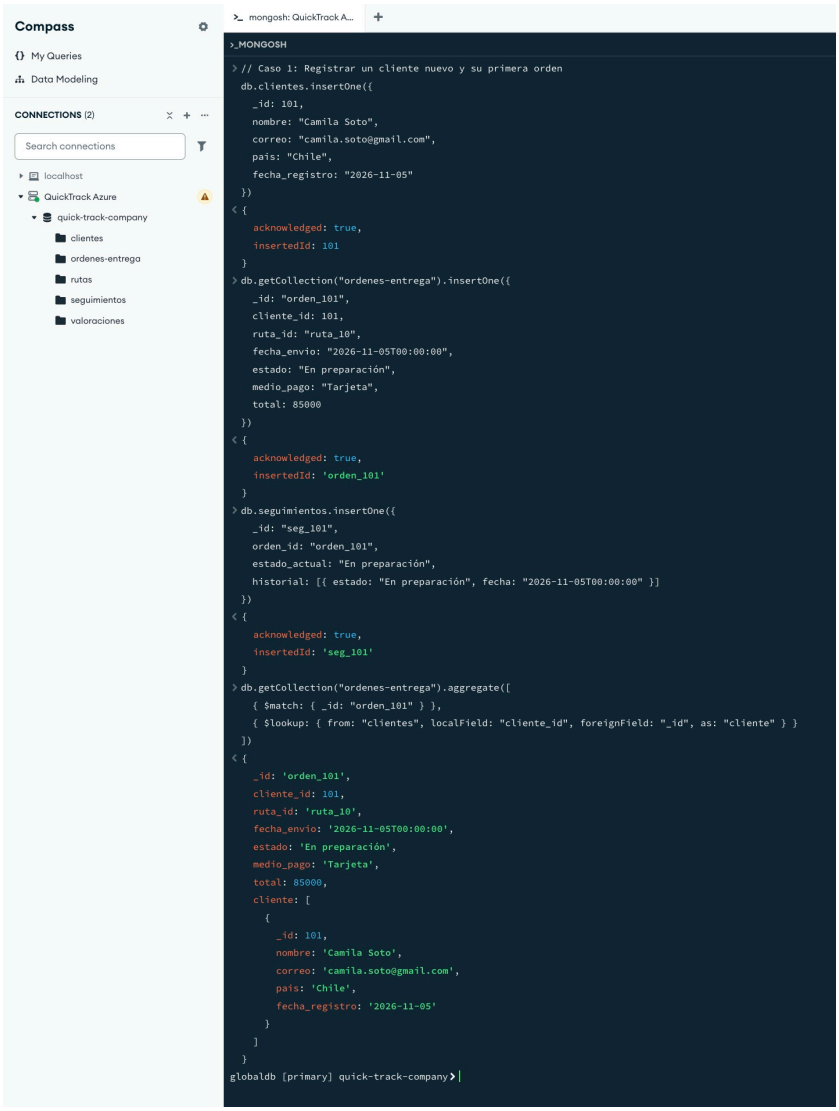
Con un reintento por el error 16500, no quedan seguimientos con diferencias.

El estado de la orden sigue sin coincidir con el del seguimiento en la mayoría de los casos. No lo cambiamos, porque no se sabe cuál de los dos es el correcto.

5.4 Caso 1: Registrar un cliente nuevo y su primera orden

Un cliente nuevo hace su primera compra. Se insertan tres documentos:

- El cliente, con _id 101.
- La orden, con cliente_id numérico.
- El seguimiento, con el primer estado en el historial.

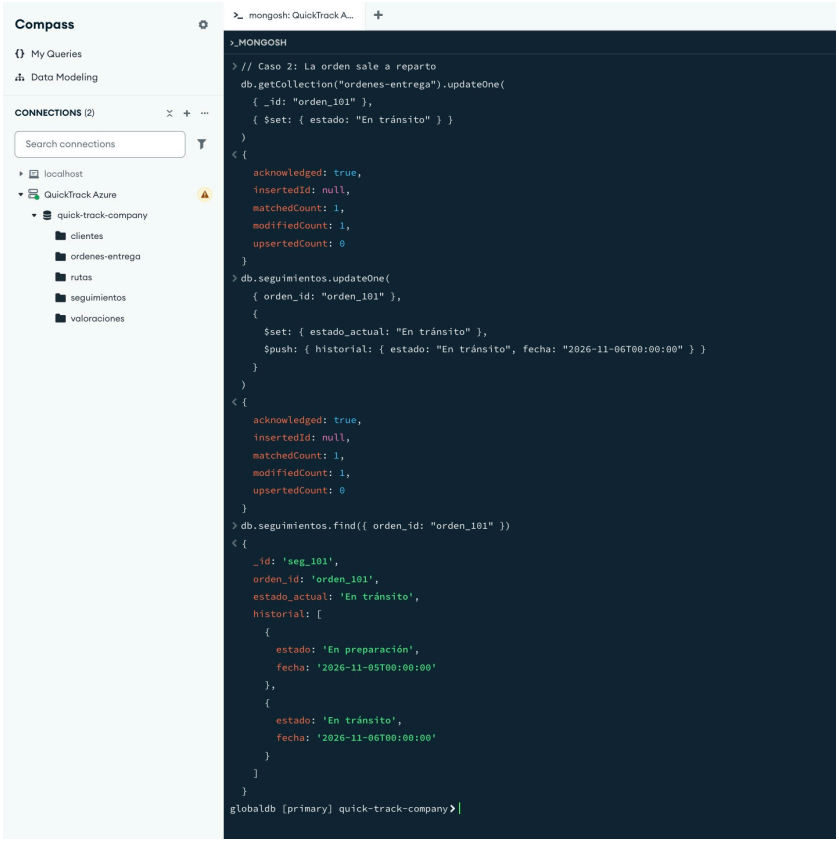


El $lookup muestra la orden con su cliente, así que quedaron bien relacionados.

5.5 Caso 2: La orden sale a reparto

Se actualizan la orden y el seguimiento juntos, para que no queden distintos:

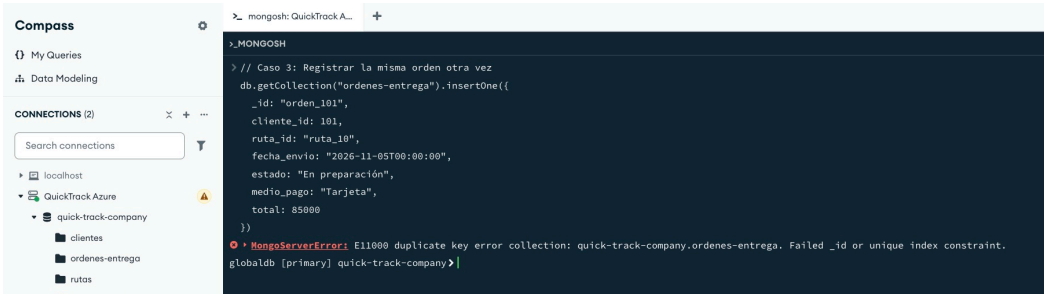
- Orden: updateOne con $set cambia el estado a "En tránsito".
- Seguimiento: $set cambia el estado_actual y $push agrega el estado al historial.



El historial queda con dos estados y coincide con la orden.

5.6 Caso 3: Registrar la misma orden otra vez

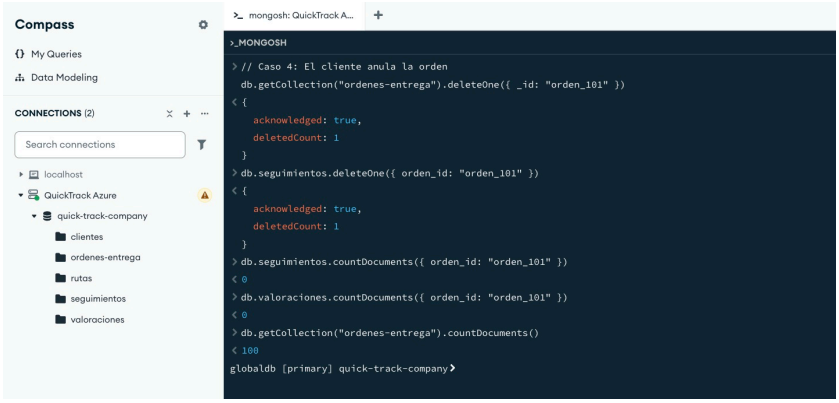
Se intenta registrar otra vez la orden 101.



MongoDB la rechaza con el error E11000, porque el _id ya existe. Así no se pueden duplicar órdenes.

5.7 Caso 4: El cliente anula la orden

Se borran la orden y su seguimiento con deleteOne, para no dejar datos sueltos.



No quedan seguimientos ni valoraciones de la orden, y vuelven a ser 100 órdenes. El cliente sigue registrado.

En la operación diaria sería mejor marcar la orden como "Cancelado", como las 23 canceladas de los datos, para no perder su historial.

6. Integración con Python

La integración se hizo en un notebook de Google Colab con la librería pymongo. El notebook se entrega junto a este informe.

6.1 Conexión desde Colab

- La cadena de conexión se guardó en los Secrets de Colab como COSMOS_URI, y el código la lee con userdata.get. Así no queda escrita en el notebook.
- La conexión se hace con MongoClient y se prueba con ping.
- Se cuentan los documentos de cada colección para confirmar que es la misma base.

```
# La cadena de conexión está guardada en los Secrets de Colab
uri = userdata.get("COSMOS_URI")

try:
    client = MongoClient(uri)
    client.admin.command("ping")
    db = client["quick-track-company"]
    logging.info("Conexión exitosa con Azure Cosmos DB")
    for nombre in sorted(db.list_collection_names()):
        print(nombre, ":", db[nombre].count_documents({}), "documentos")
except PyMongoError as e:
    logging.error(f"No se pudo conectar: {e}")
```

```
/tmp/ipykernel_10158/1381193139.py:5: UserWarning: You appear to be connected to a CosmosDB cluster. For more information regarding feature
compatibility and support please visit https://www.mongodb.com/supportability/cosmosdb
  client = MongoClient(uri)
2026-11-05 20:57:20,103 | INFO | Conexión exitosa con Azure Cosmos DB
clientes : 101 documentos
ordenes-entrega : 100 documentos
rutas : 100 documentos
seguimientos : 100 documentos
valoraciones : 100 documentos
```

pymongo muestra un aviso porque la base es Cosmos DB, igual que Compass. No es un error. La colección clientes tiene 101 documentos por el cliente agregado en el apartado 5.4.

6.2 Funciones CRUD

- crear_orden: valida la orden, la inserta y crea su seguimiento.
- buscar_ordenes: busca con un filtro y devuelve solo cliente_id, estado y total.
- actualizar_estado: cambia el estado de la orden y agrega el cambio al historial del seguimiento, como en el apartado 5.5.
- eliminar_orden: borra la orden y su seguimiento.

Probamos las funciones con una orden nueva del cliente 101:

```
nueva_orden = {
    "_id": "orden_102",
    "cliente_id": 101,
    "ruta_id": "ruta_25",
    "fecha_envio": "2026-11-06T00:00:00",
    "estado": "En preparación",
    "medio_pago": "Transferencia",
    "total": 64990
}

crear_orden(nueva_orden)

for orden in buscar_ordenes({"cliente_id": 101}):
    print(orden)

actualizar_estado("orden_102", "Despachado", "2026-11-07T00:00:00")
print(seguimientos.find_one({"orden_id": "orden_102"}))
```

```
2026-11-05 21:03:43,050 | INFO | Orden creada: orden_102
2026-11-05 21:03:43,088 | INFO | Búsqueda {'cliente_id': 101}, órdenes encontradas: 1
{'_id': 'orden_102', 'cliente_id': 101, 'estado': 'En preparación', 'total': 64990}
2026-11-05 21:03:43,184 | INFO | Orden orden_102 actualizada a Despachado
{'_id': 'seg_102', 'orden_id': 'orden_102', 'estado_actual': 'Despachado', 'historial': [{'estado': 'En preparación', 'fecha': '2026-11-
06T00:00:00'}, {'estado': 'Despachado', 'fecha': '2026-11-07T00:00:00'}]}
```

La orden se creó, la búsqueda la encontró y el seguimiento quedó con dos estados. Al final la borramos con eliminar_orden, como se ve en el log del apartado 7.3.

7. Trazabilidad y control de errores

7.1 Registro de operaciones

- Usamos la librería logging de Python.
- Cada mensaje tiene fecha, hora, nivel y descripción.
- INFO: operaciones exitosas.
- WARNING: datos rechazados u órdenes que no existen.
- ERROR: errores de la base, como una orden duplicada.
- Los mensajes se muestran en pantalla y se guardan en el archivo quicktrack.log.

7.2 Validaciones y manejo de errores

- Antes de insertar, validar_orden revisa que estén todos los campos, que el estado y el medio de pago sean válidos, que el total sea mayor a 0 y que el cliente exista.
- Cada función usa try y except. Si algo falla, se registra en el log y el programa sigue.

Probamos seis errores a propósito:

```
# Orden repetida
crear_orden(nueva_orden)

# Falta un campo
crear_orden({"_id": "orden_103", "cliente_id": 101, "total": 20000})

# Total negativo
crear_orden({
    "_id": "orden_103",
    "cliente_id": 101,
    "ruta_id": "ruta_25",
    "fecha_envio": "2026-11-06T00:00:00",
    "estado": "En preparación",
    "medio_pago": "Efectivo",
    "total": -5000
})

# Cliente que no existe
crear_orden({
    "_id": "orden_103",
    "cliente_id": 999,
    "ruta_id": "ruta_25",
    "fecha_envio": "2026-11-06T00:00:00",
    "estado": "En preparación",
    "medio_pago": "Efectivo",
    "total": 20000
})

# Estado que no existe
actualizar_estado("orden_102", "Perdido", "2026-11-07T00:00:00")

# Orden que no existe
actualizar_estado("orden_999", "Entregado", "2026-11-07T00:00:00")
```

```
2026-11-05 21:03:52,242 | ERROR | La orden orden_102 ya existe
2026-11-05 21:03:52,244 | WARNING | Orden orden_103 rechazada: falta el campo ruta_id
2026-11-05 21:03:52,246 | WARNING | Orden orden_103 rechazada: el total debe ser mayor a 0
2026-11-05 21:03:52,295 | WARNING | Orden orden_103 rechazada: no existe el cliente 999
2026-11-05 21:03:52,296 | WARNING | Estado no válido: Perdido
2026-11-05 21:03:52,327 | WARNING | No existe la orden orden_999
```

Ninguno detuvo el programa y todos quedaron registrados.

7.3 Archivo de log

El archivo quicktrack.log guarda todas las operaciones en orden y con su hora. Así se puede revisar qué se hizo y cuándo.

```
!cat quicktrack.log
```

```
2026-11-05 20:57:20,103 | INFO | Conexión exitosa con Azure Cosmos DB
2026-11-05 21:03:43,050 | INFO | Orden creada: orden_102
2026-11-05 21:03:43,088 | INFO | Búsqueda {'cliente_id': 101}, órdenes encontradas: 1
2026-11-05 21:03:43,184 | INFO | Orden orden_102 actualizada a Despachado
2026-11-05 21:03:52,242 | ERROR | La orden orden_102 ya existe
2026-11-05 21:03:52,244 | WARNING | Orden orden_103 rechazada: falta el campo ruta_id
2026-11-05 21:03:52,246 | WARNING | Orden orden_103 rechazada: el total debe ser mayor a 0
2026-11-05 21:03:52,295 | WARNING | Orden orden_103 rechazada: no existe el cliente 999
2026-11-05 21:03:52,296 | WARNING | Estado no válido: Perdido
2026-11-05 21:03:52,327 | WARNING | No existe la orden orden_999
2026-11-05 21:04:00,098 | INFO | Orden orden_102 eliminada junto con su seguimiento
2026-11-05 21:04:00,128 | WARNING | No existe la orden orden_102
```

Las horas están en UTC, que es la hora del servidor de Colab.